

Data Structures (CS2103)

AVL Tree

Prepared By – Ashish K Layek
CST, IEST Shibpur

Background

□ Recall:

- Binary search trees store linearly ordered data
- Best case height: $\mathbf{O}(\ln(n))$
- Worst case height: $\mathbf{O}(n)$

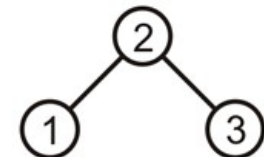
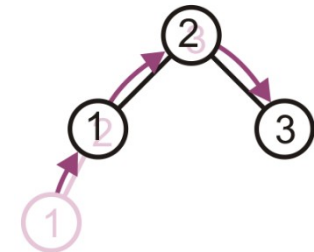
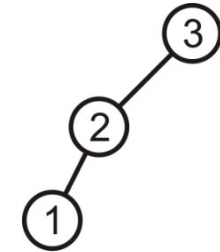
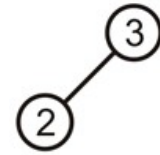
❖ Requirement of Balance:

- Define and maintain a *balance* to ensure $\mathbf{O}(\ln(n))$ height
-

Prototypical Examples 1

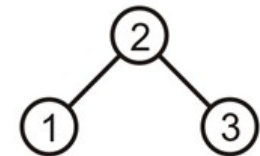
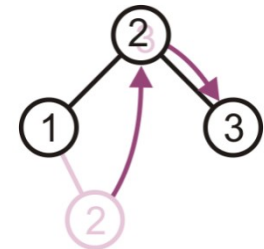
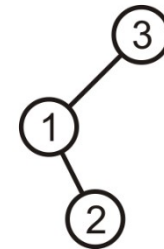
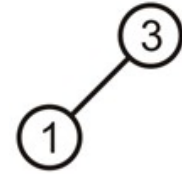
❑ Examples demonstrate how we can correct for imbalances:

- Starting with this tree, add 1:
- This is more like a linked list; however, we can fix this...
- Promote 2 to the root, demote 3 to be 2's right child, and 1 remains the left child of 2
- The result is a perfect, though trivial tree



Prototypical Examples 2

- Alternatively, given this tree, insert 2
 - Again, the product is a linked list; however, we can fix this, too
 - Promote 2 to the root, and assign 1 and 3 to be its children
 - The result is again a perfect, though trivial tree
- ❑ These examples may seem trivial, but they are the basis for the corrections in the AVL trees



AVL Trees

- ❑ We will focus on the first strategy: AVL trees
 - Named after **A**delson-**V**elskii and **L**andis

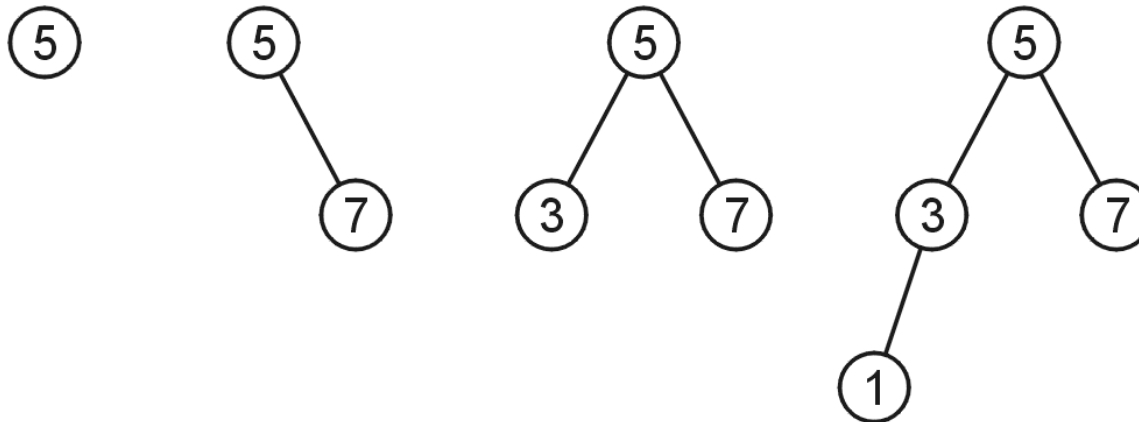
 - ❑ Balance is defined by comparing the height of the two sub-trees

 - ❑ Recall:
 - An empty tree has height -1
 - A tree with a single node has height 0
-

AVL Trees

- ❑ A binary search tree is said to be AVL balanced if:
 - The difference in the heights between the left and right sub-trees is at most 1, and
 - Both sub-trees are themselves AVL trees

❖ Example: AVL trees with 1, 2, 3, and 4 nodes:



AVL Trees: Definitions

- ❑ Definition: An empty binary tree is height balanced. If T is a nonempty binary tree with T_L and T_R as its left and right subtrees, then T is height balanced iff
 - T_L and T_R are height balanced, and
 - $|h_L - h_R| \leq 1$ where h_L and h_R are the heights of T_L and T_R respectively.

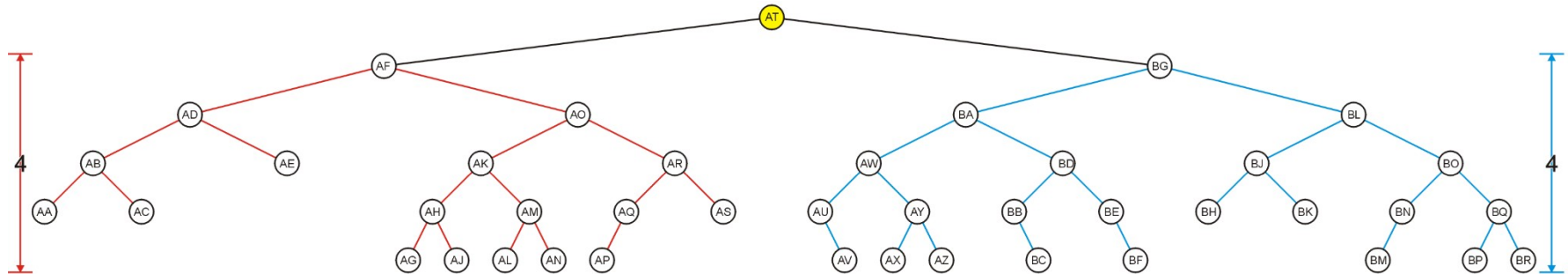
 - ❑ Definition: The *balance factor*, $BF(T)$, of a node, T , in a binary tree is defined as $(h_L - h_R)$, where h_L and h_R are, respectively, the heights of the left and right subtrees of T . For any node T in an AVL tree $BF(T) \in \{-1, 0, 1\}$.
-

AVL Trees

❖ Example of a larger AVL tree (42 nodes):

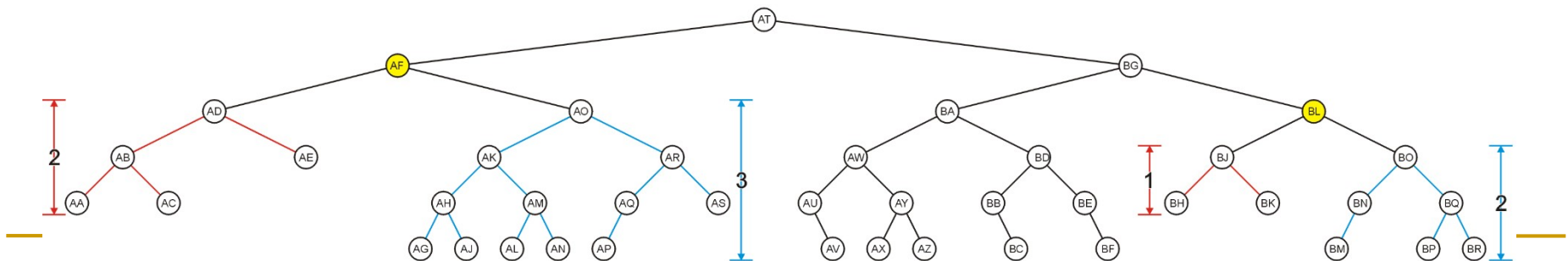
❑ The root node is AVL-balanced:

➤ Both sub-trees are of height 4:



❑ All other nodes (e.g., AF and BL) are AVL balanced

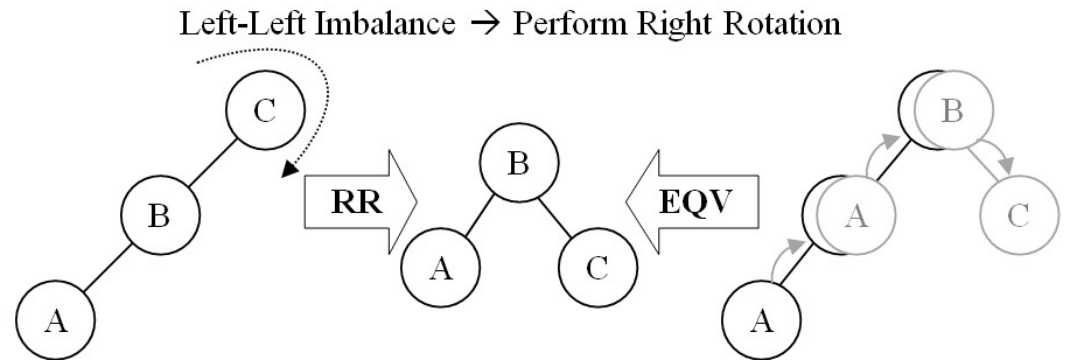
➤ The sub-trees differ in height by at most one



Balancing Fundamental

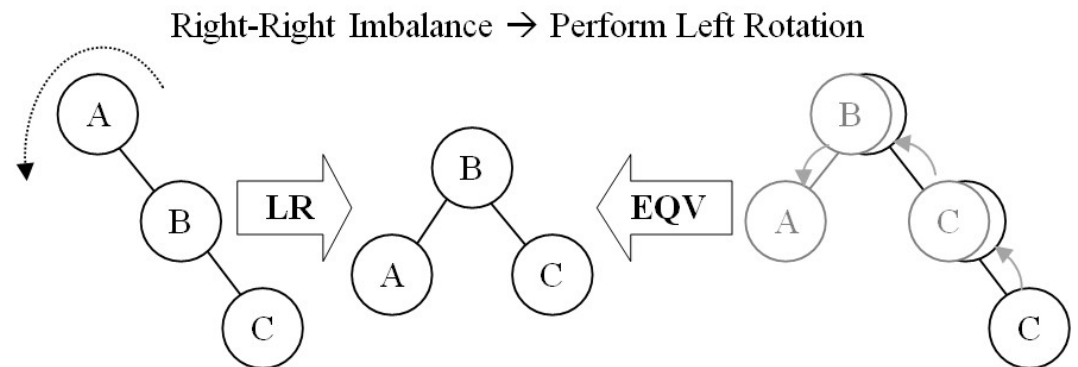
- ❑ If imbalance found at a node and in the path involved is **left and left** subtree

➤ Perform **Right Rotation**



- ❑ If imbalance found at a node and in the path involved is **right and right** subtree

➤ Perform **Left rotation**



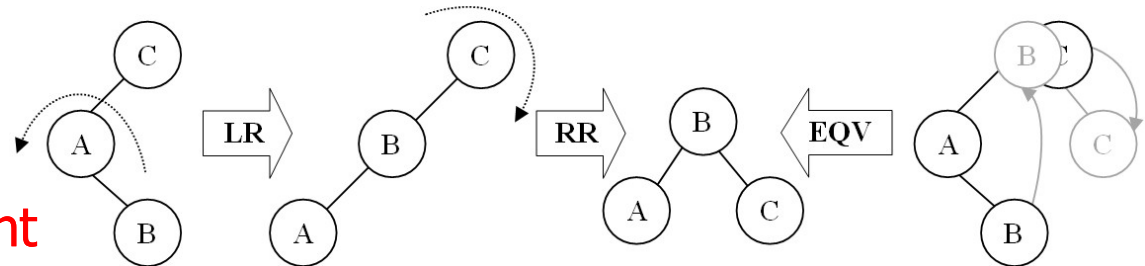
Balancing Fundamental

Contd

- ❑ If imbalance found at a node and in the involved path is **left and right** subtree

- Perform **Left then Right Rotation**

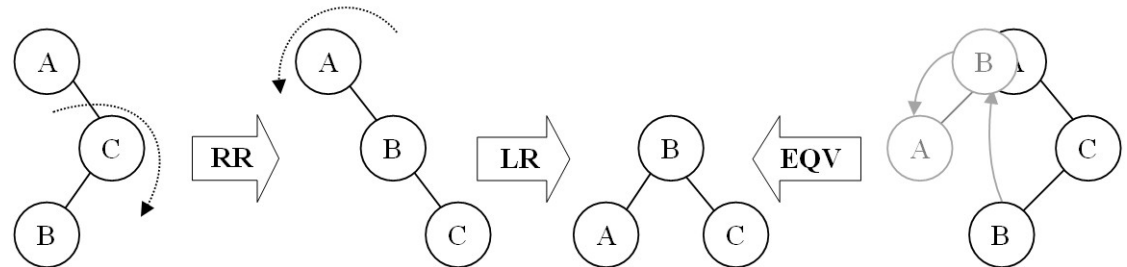
Left-Right Imbalance → Perform Left then Right Rotation



- ❑ If imbalance found at a node and in the involved path is **right and left** subtree

- Perform **Right then Left Rotation**

Right-Left Imbalance → Perform Right then Left Rotation



EQV: Equivalent

Maintaining Balance

- ❑ To maintain AVL balance, observe that:
 - Inserting a node can increase the height of a tree by at most 1
 - Removing a node can decrease the height of a tree by at most 1

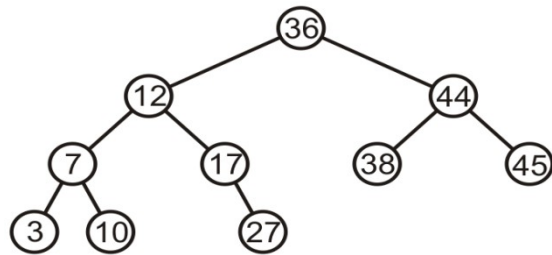
 - ❑ To calculate changes in height, the it must run in $O(1)$ time.
 - Therefore, in the tree node structure, introduce a member variable `int tree_height`
 - It should be calculated and set during insertion or deletion.
-

Maintaining Balance: Insertion

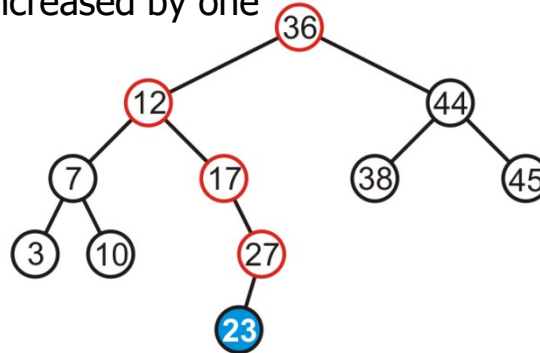
- ❑ If a tree is AVL balanced, for an insertion to cause an imbalance:
 - The heights of the sub-trees must differ by 1
 - The insertion must increase the height of the deeper sub-tree by 1
-

Maintaining Balance: Insertion Example

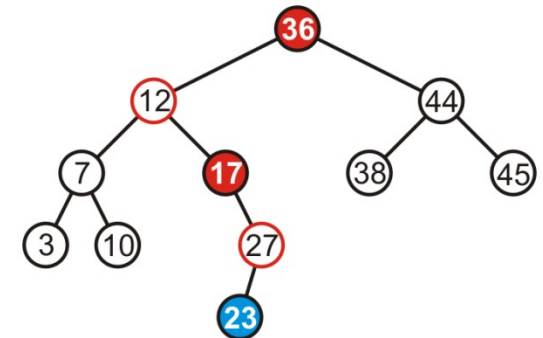
Suppose we insert 23 into our initial tree



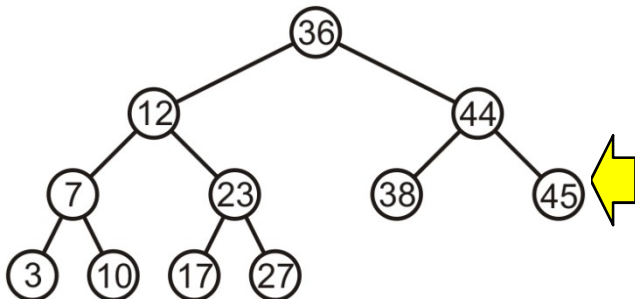
The heights of each of the subtrees from here to the root are increased by one



However, only two of the nodes are unbalanced: 17 and 36

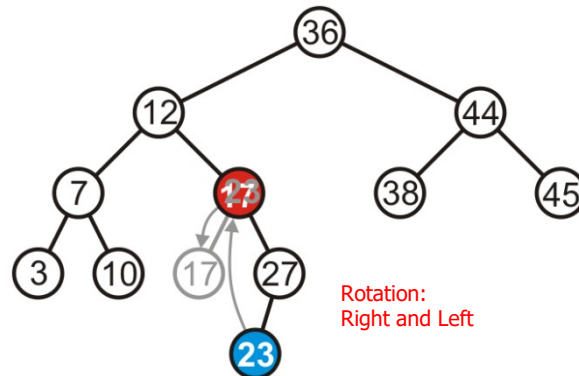


Thus, that node is no longer unbalanced



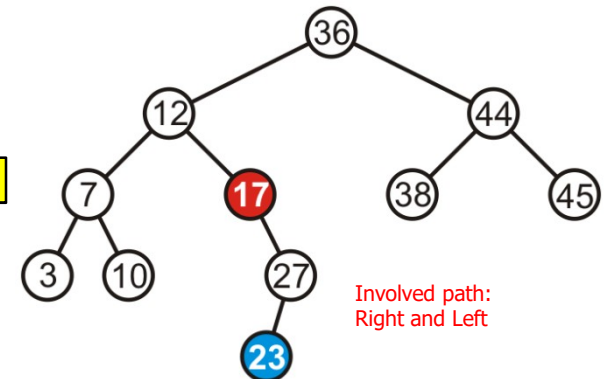
Incidentally, neither is the root now balanced again, too

We can promote 23 to where 17 is, and make 17 the left child of 23



Rotation:
Right and Left

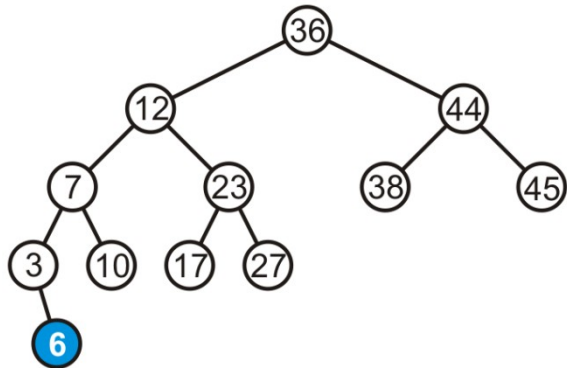
We only have to fix the imbalance at the lowest node, at 17



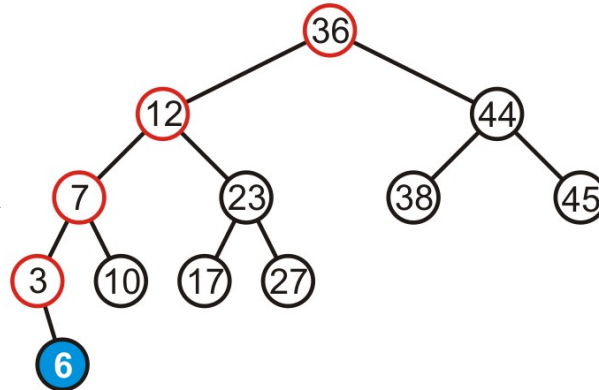
Involved path:
Right and Left

Maintaining Balance: Insertion Example 2

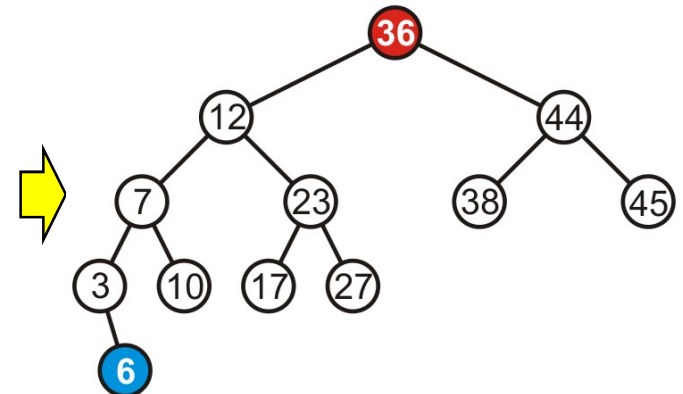
Consider insert 6 into the tree



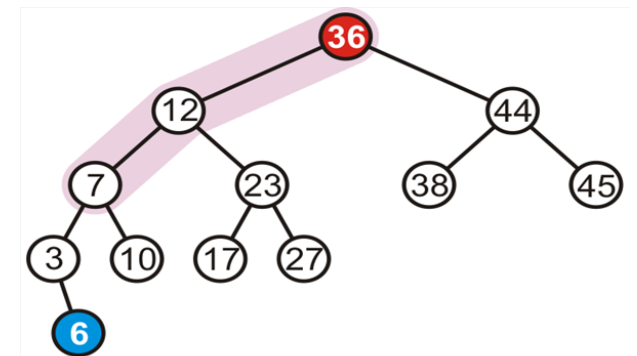
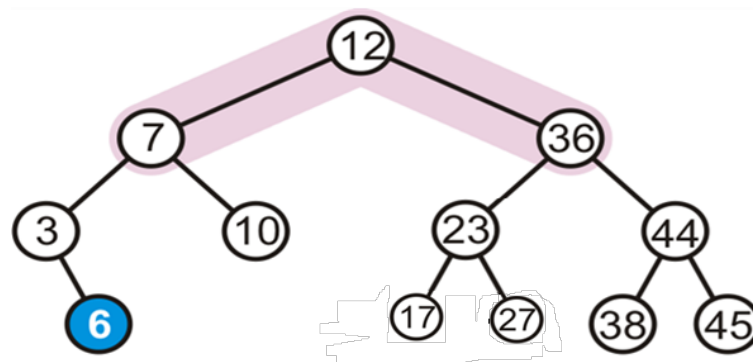
The heights of each of the sub-trees from here to the root are



However, only the root node is now unbalanced



But HOW ???

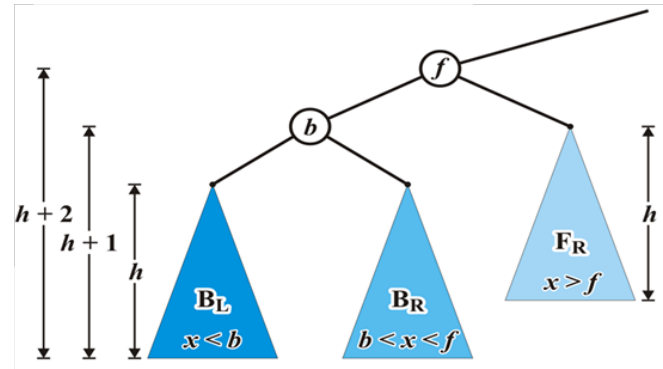


To understand this, we will look at the general case...

Maintaining Balance: Case 1

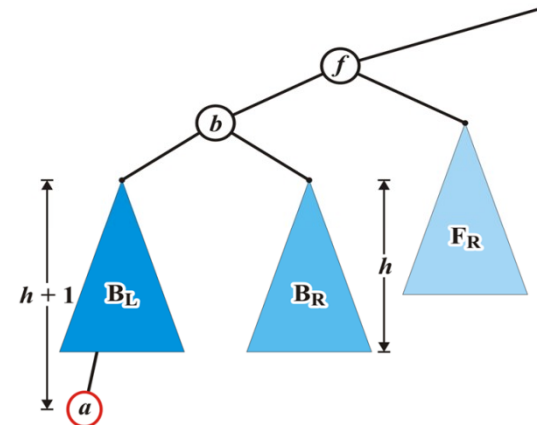
Consider the following setup.

- ❖ Each blue triangle represents a tree of height h



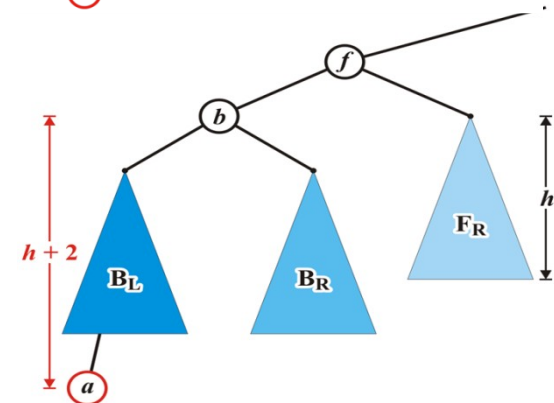
Insert a into this tree: it falls into the left subtree B_L of b

- ❖ Assume B_L remains balanced
- ❖ Thus, the tree rooted at b is also balanced



The tree rooted at node f is now unbalanced

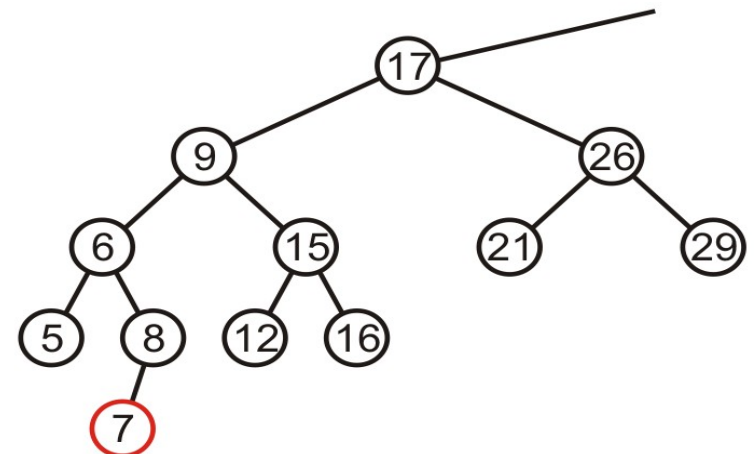
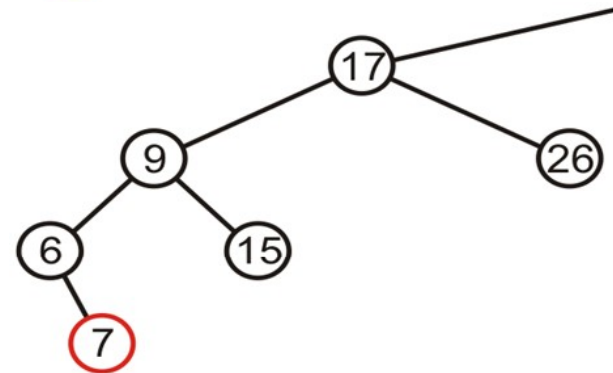
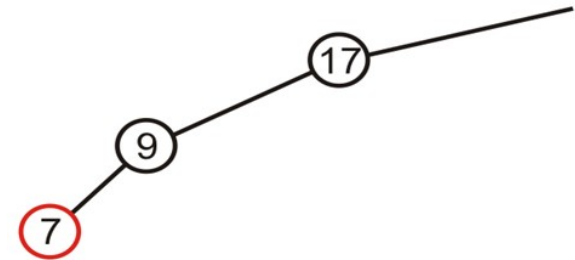
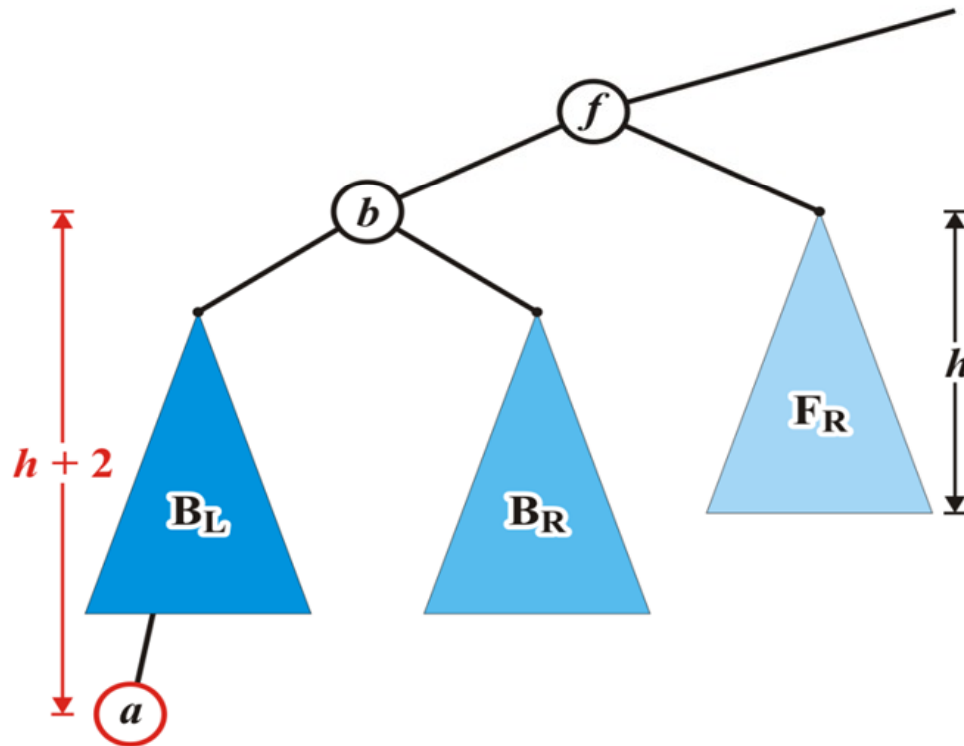
- ❖ We need to correct the imbalance at this node



➤ This is a case of Left-Left imbalance

Maintaining Balance: Case 1

Here are examples of when the insertion of 7 may cause this situation when $h = -1, 0$, and 1



Maintaining Balance: Case 1

❑ Modify these three pointers

- ❖ At this point, the unbalanced root node is f

❑ Specifically, we will rotate these two nodes around the root:

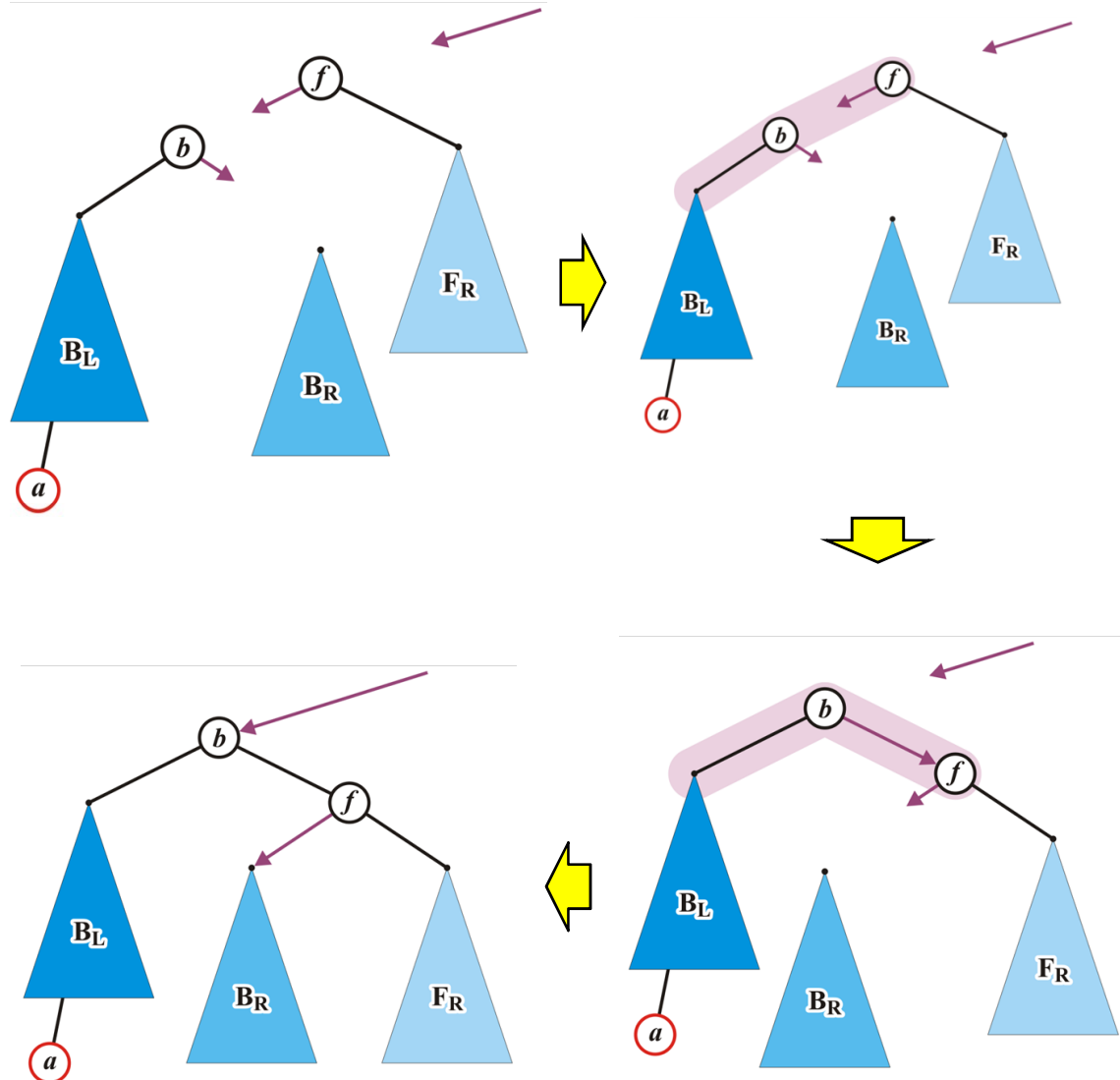
- ❖ Recall the first prototypical example
- ❖ Promote node b to the root and demote node f to be the right child of b
- ❖ *This is equivalent of Right Rotation*

```
treeNode* rightRotate(treeNode * f){
    treeNode * b = f->left;
    treeNode * bl = b->left;
    treeNode * br = b->right;

    b->right = f;
    f->left = br;

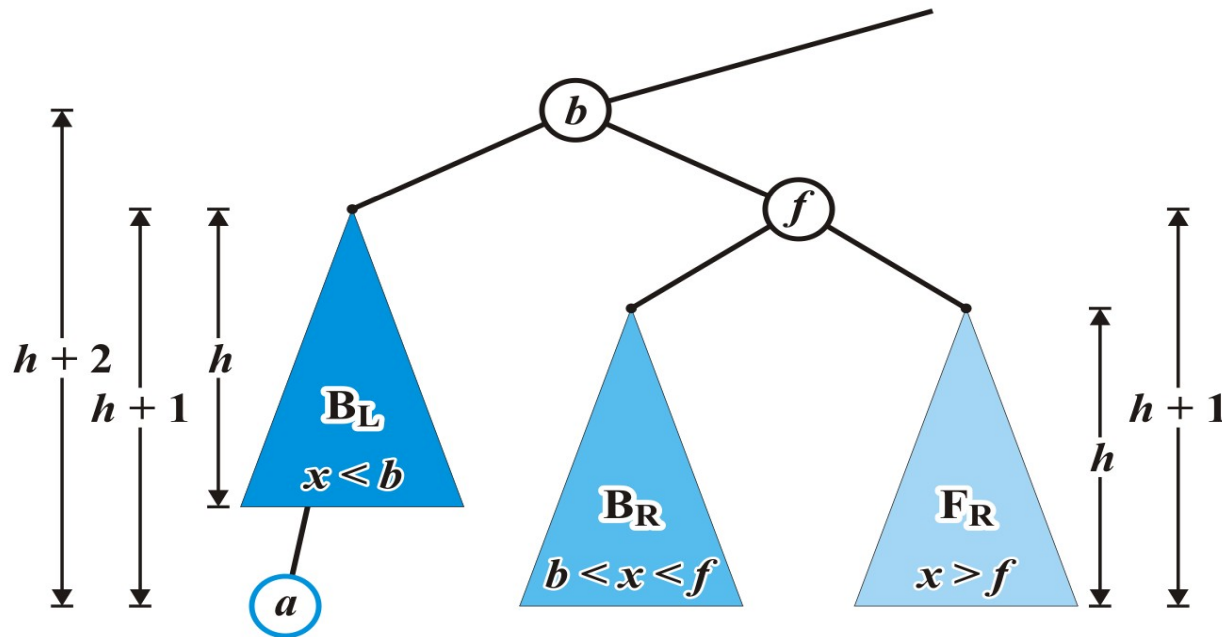
    // TODO: Set heights of subtrees

    return b;
}
```



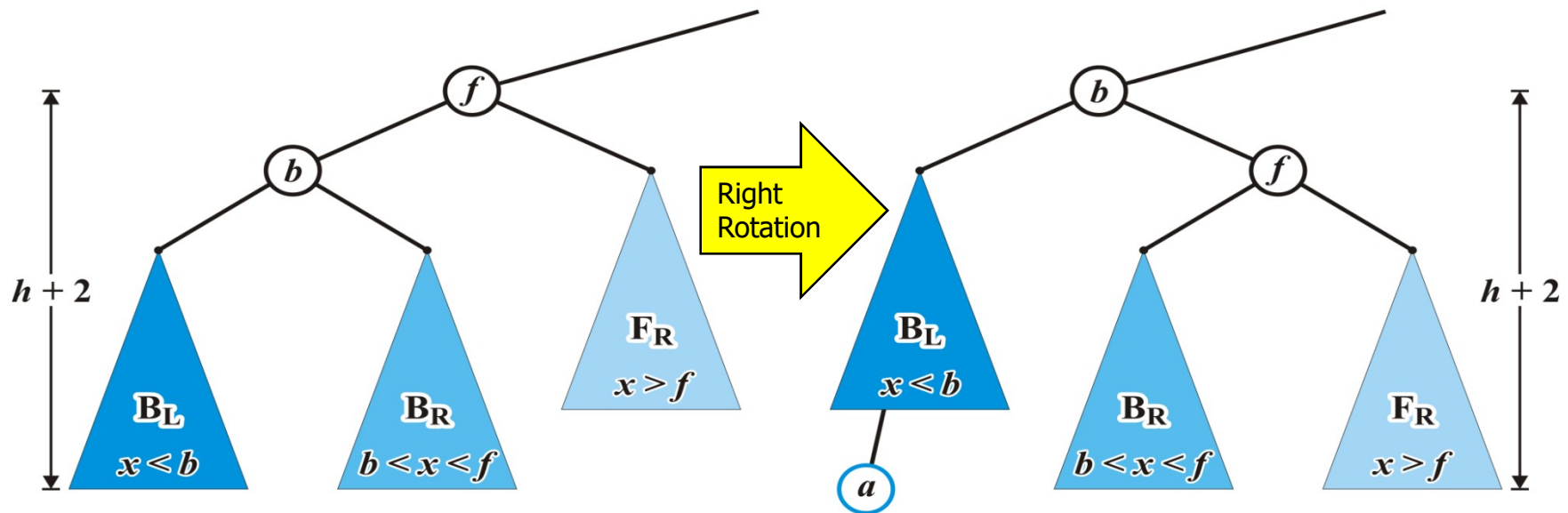
Maintaining Balance: Case 1

- The nodes ***b*** and ***f*** are now balanced and all remaining nodes of the subtrees are in their correct positions
 - The height of ***f*** is now $h + 1$ while ***b*** remains at height $h + 2$



Maintaining Balance: Case 1

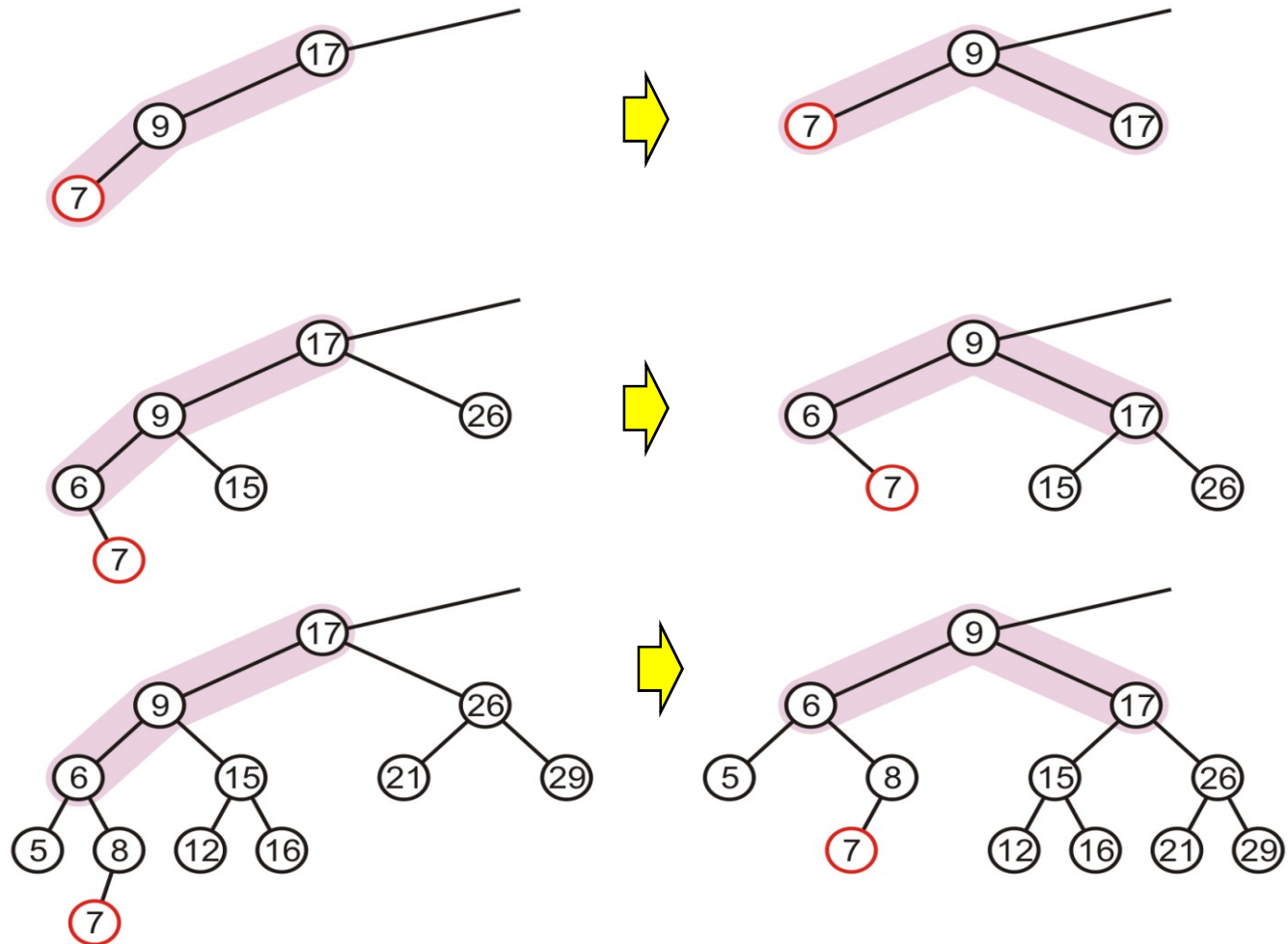
- Additionally, height of the corrected tree rooted at ***b*** equals the original height of the tree rooted at ***f***
 - Thus, this insertion will no longer affect the balance of any ancestors all the way back to the root.
 - That's why we only have to fix the imbalance at the lowest node



Note: In case of right-right imbalance, left rotation works using same idea

Maintaining Balance: Case 1

In our three sample cases with $h = -1, 0$, and 1 , the node is now balanced and the same height as the tree before the insertion



Maintaining Balance: Case 2

Consider the following setup

- ❖ Each blue triangle represents a tree of height h

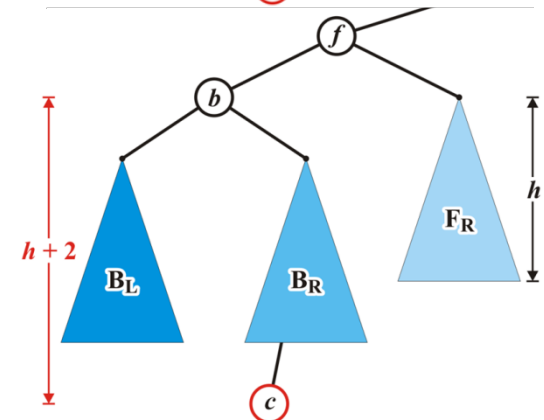
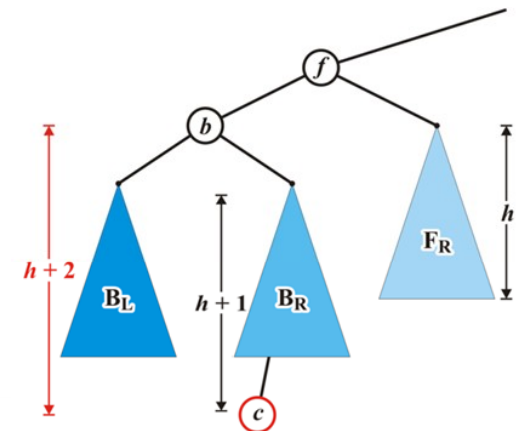
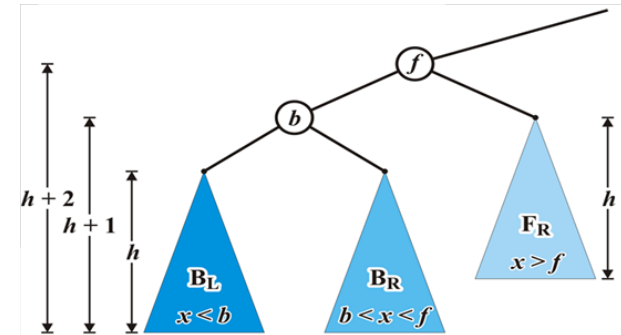
Alternatively, consider the insertion of c where $b < c < f$ into the tree. It falls into the right subtree B_R of b

- ❖ Assume B_R remains balanced
- ❖ Thus, the tree rooted at b is also balanced

The tree rooted at node f is now unbalanced

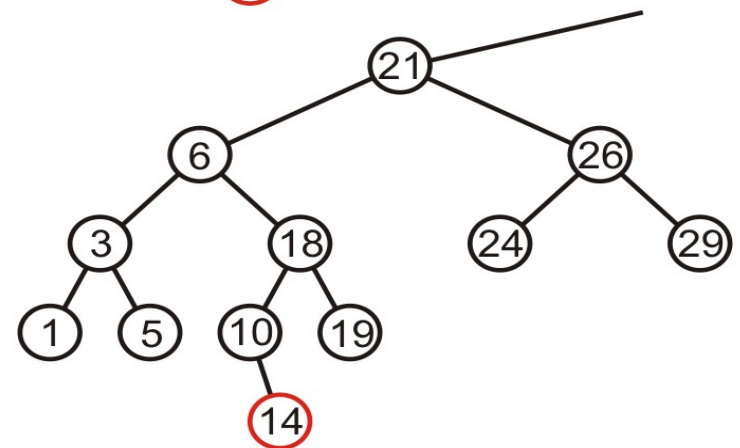
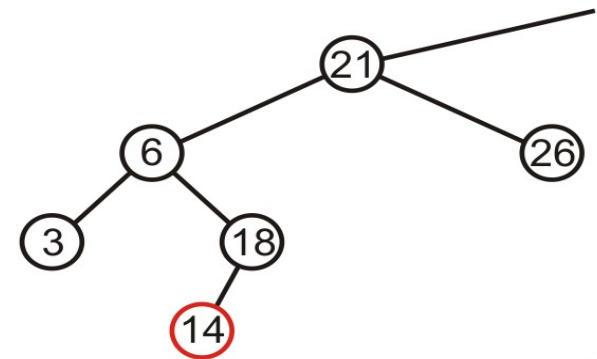
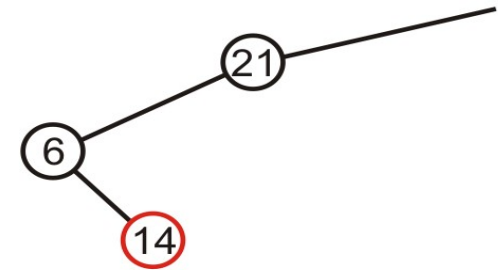
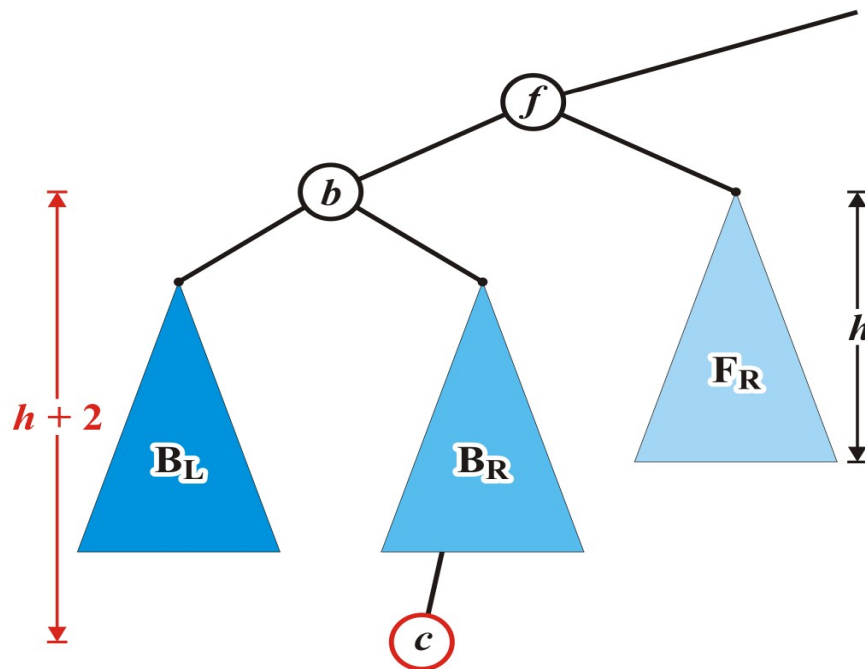
- ❖ We need to correct the imbalance at this node

➤ This is a case of Left-Right imbalance



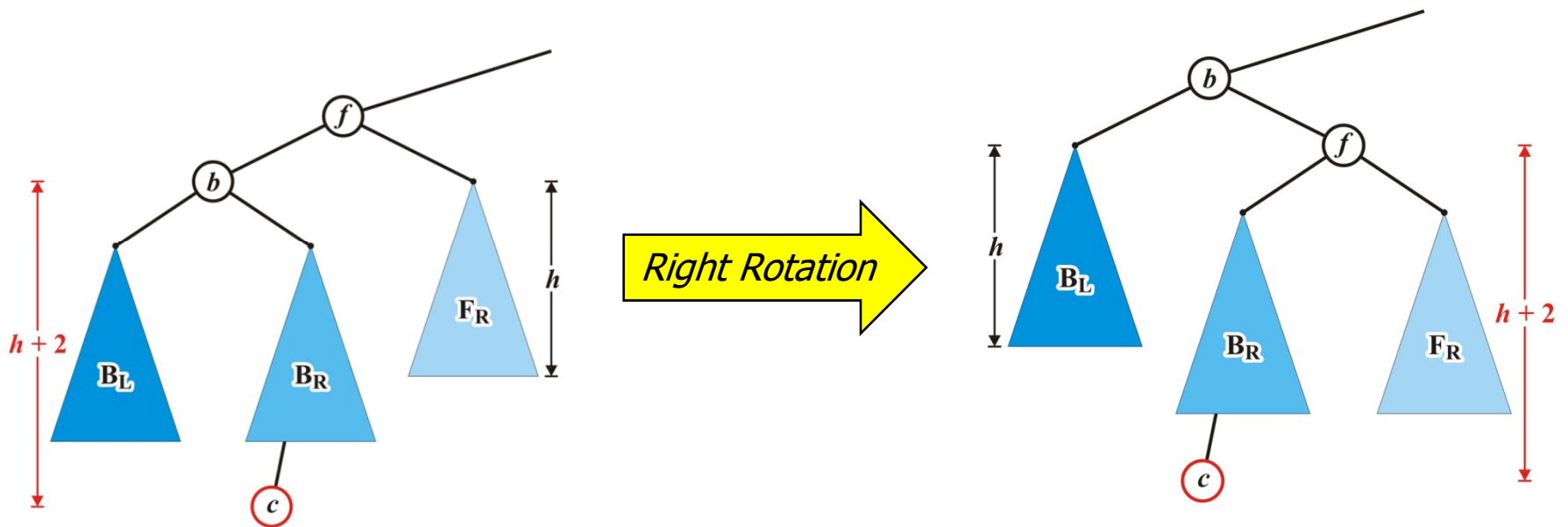
Maintaining Balance: Case 2

Here are examples of when the insertion of 14 may cause this situation when $h = -1, 0$, and 1



Maintaining Balance: Case 2

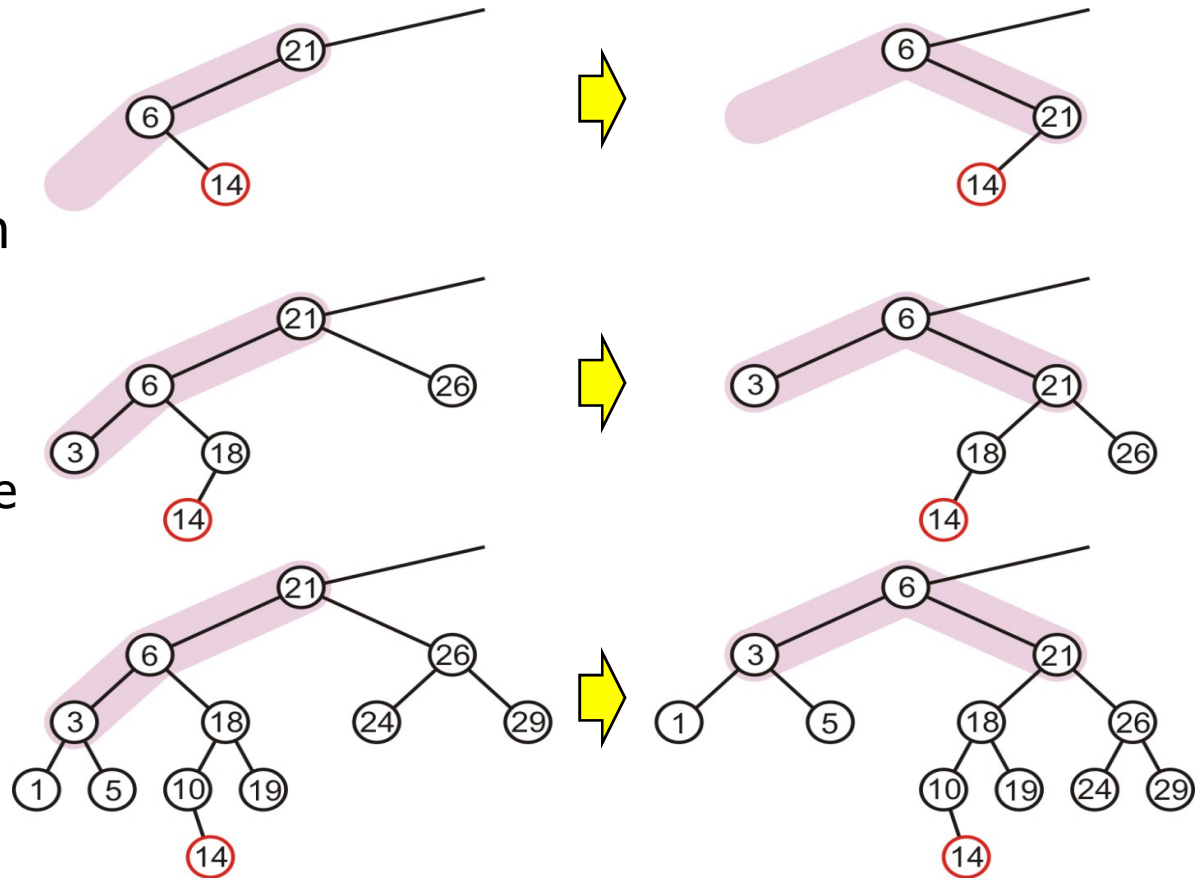
Unfortunately, the previous correction (Right Rotation) does not fix the imbalance at the root of this sub-tree: the new root, b , remains unbalanced



Maintaining Balance: Case 2

In our three sample cases with $h = -1, 0$, and 1 , doing the same thing as before results in a tree that is still unbalanced...

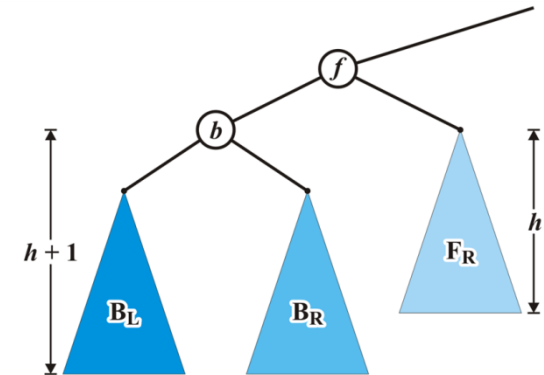
- The imbalance is just shifted to the other side



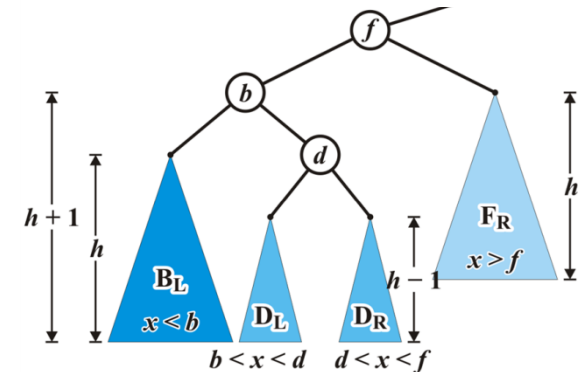
➤ So a new technique has to be applied

Maintaining Balance: Case 2

□ Original Tree



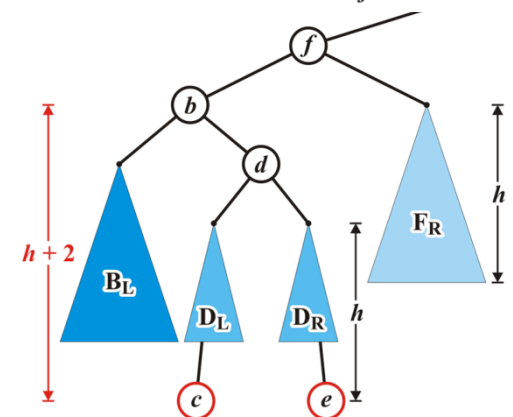
□ Re-label the tree by dividing the right subtree of b into a tree rooted at d with two subtrees of height $h-1$



❖ What if B_R is an empty tree has height -1 ?

□ Now an insertion causes an imbalance at f

❖ The addition of either c or e will cause this



Maintaining Balance: Case 2

❑ Reassign these pointers

❖ At this point, the unbalanced root node is f

❑ Specifically, we will order these three nodes as a perfect tree:

- ❖ Recall the second prototypical example
- ❖ To achieve this, b and f will be assigned as children of the new root d
- ❖ We also have to connect the two subtrees and original parent of f
- ❖ *This is equivalent of Left-Right Rotation*

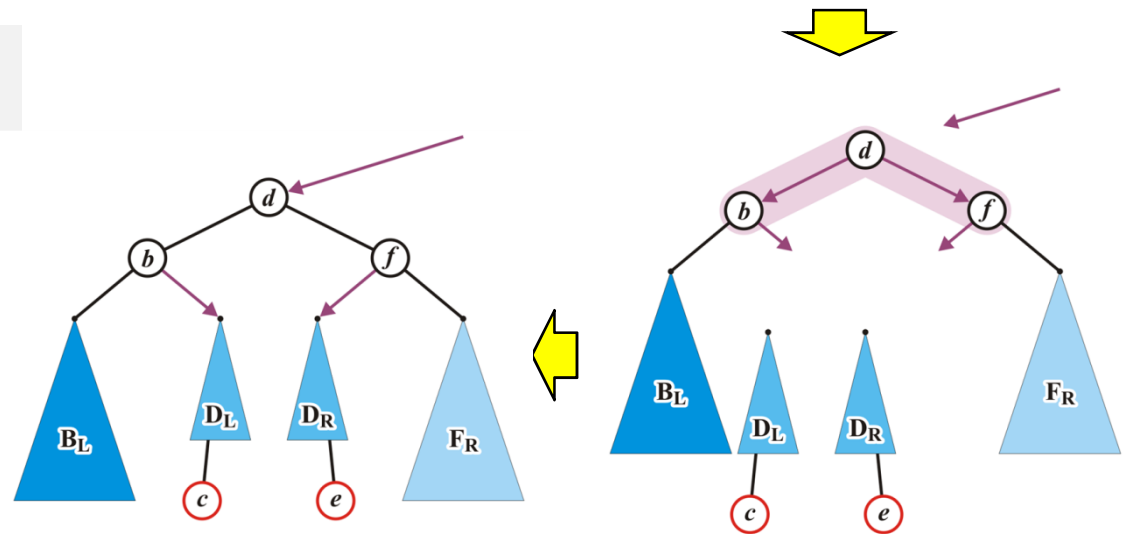
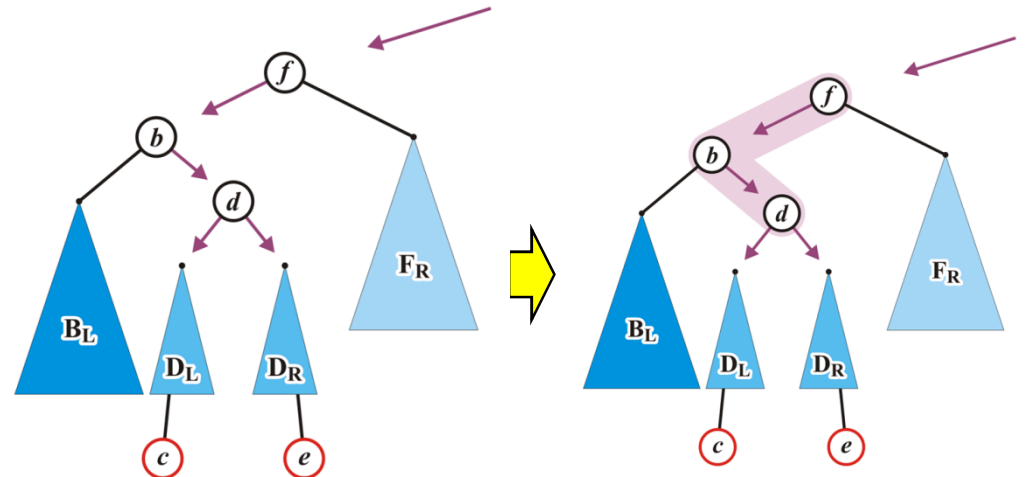
```
treenode* leftRightRotate(treenode *f){
    treenode * b = f->left;
    treenode * bl = b->left;
    treenode * d = b->right;

    b->right = d->left;
    f->left = d->right;

    d->left = b;
    d->right = f;

    // TODO: Set heights of subtrees

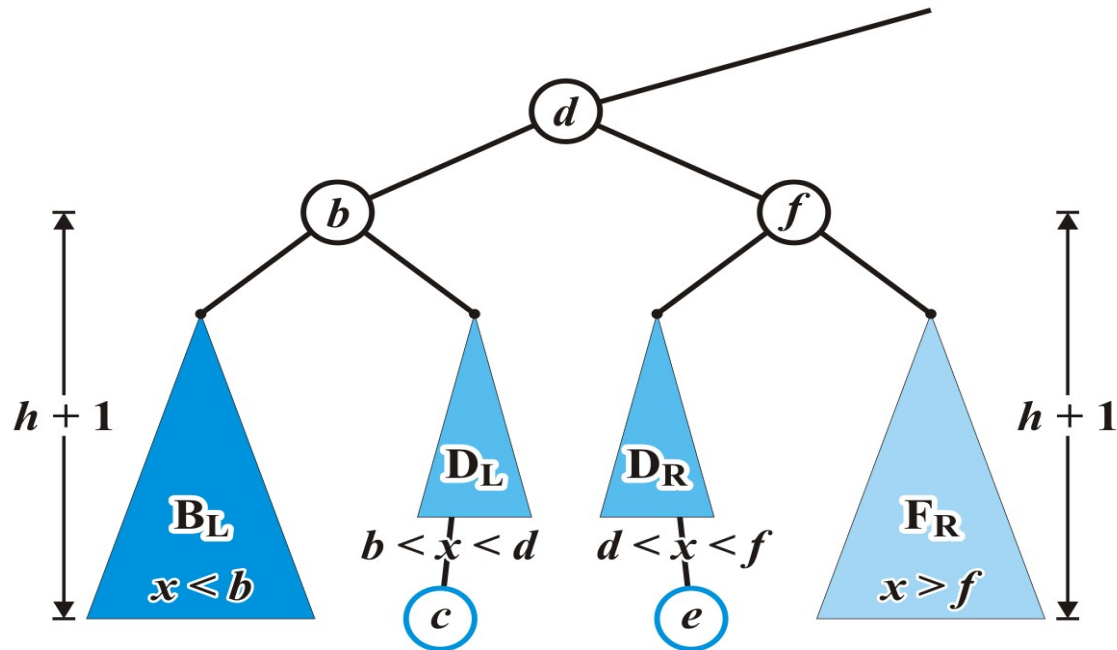
    return d;
}
```



Maintaining Balance: Case 2

Now the tree rooted at ***d*** is balanced

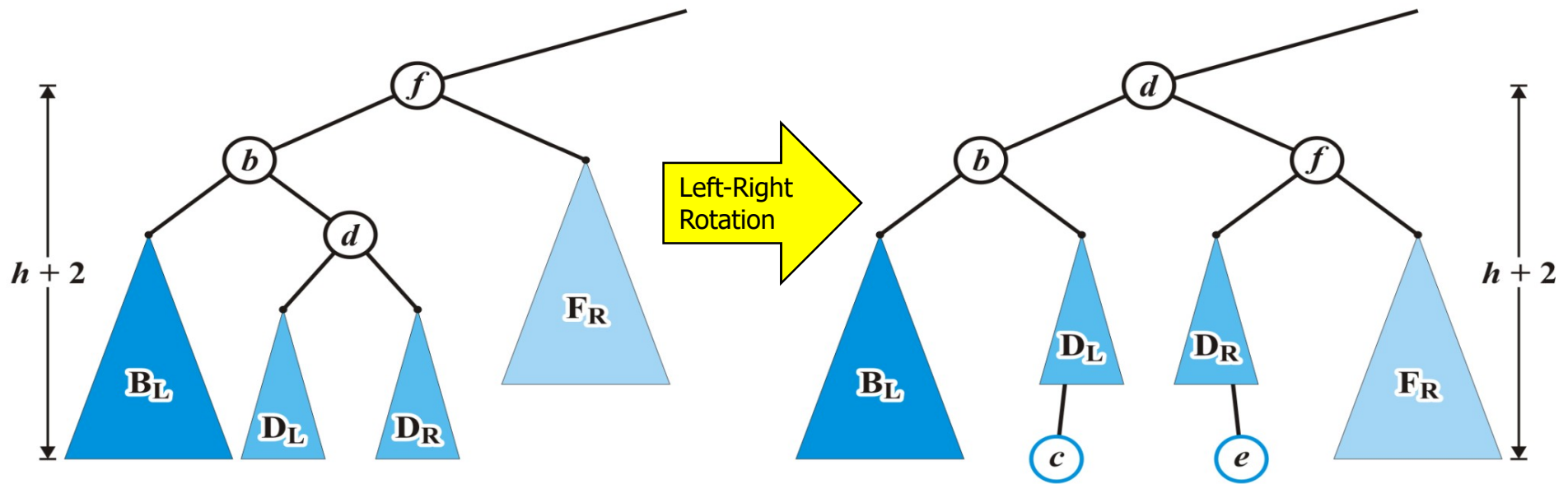
- After the correction, height of ***b*** and ***f*** become $h + 1$ and ***d*** is $h + 2$



Maintaining Balance: Case 2

Again, the height of the root did not change

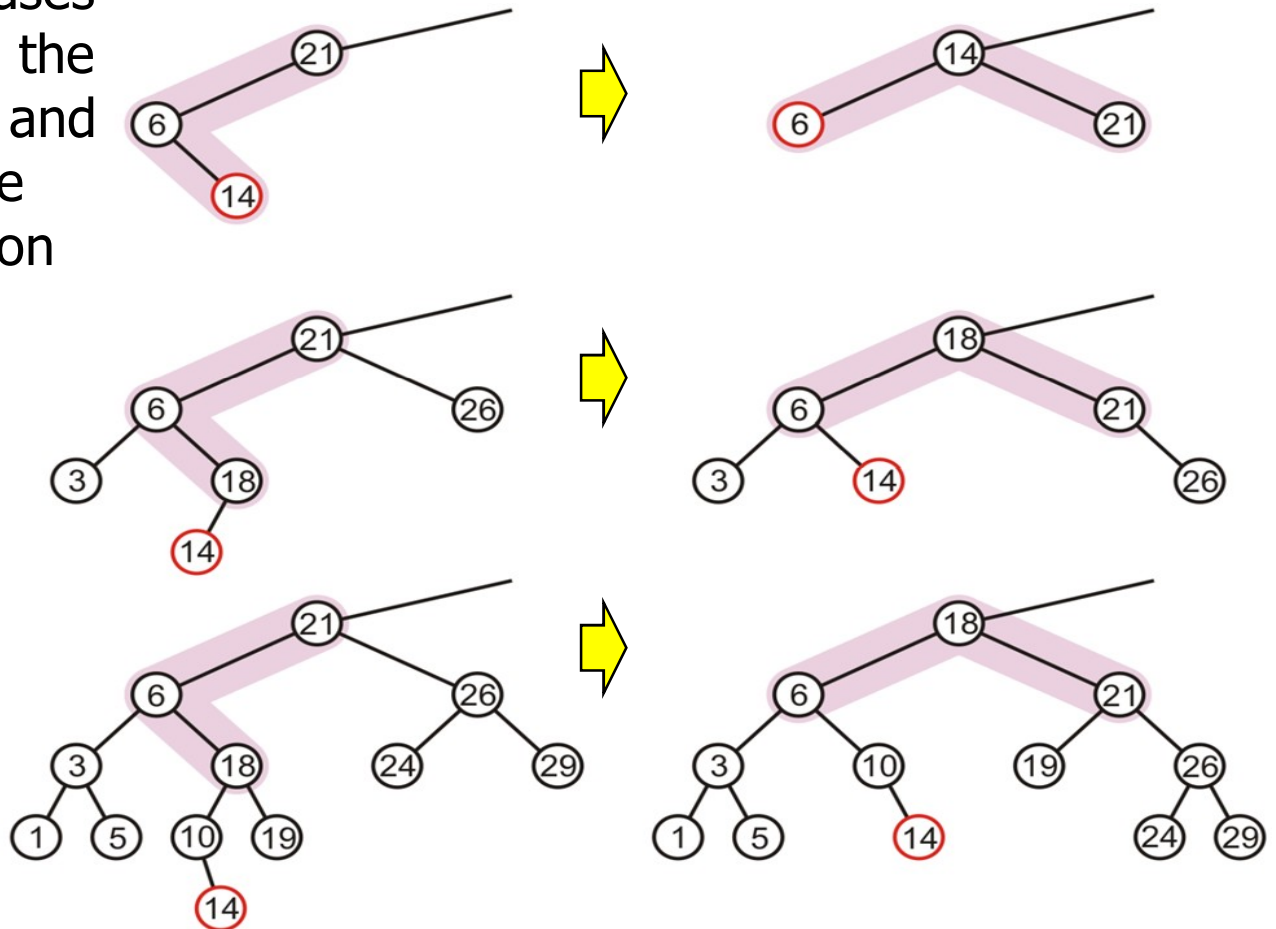
- The heights of all three nodes changed in this process



Note: Incase of right-left imbalance, right-left rotation works using same idea

Maintaining Balance: Case 2

In our three sample cases with $h = -1, 0$, and 1 , the node is now balanced and the same height as the tree before the insertion

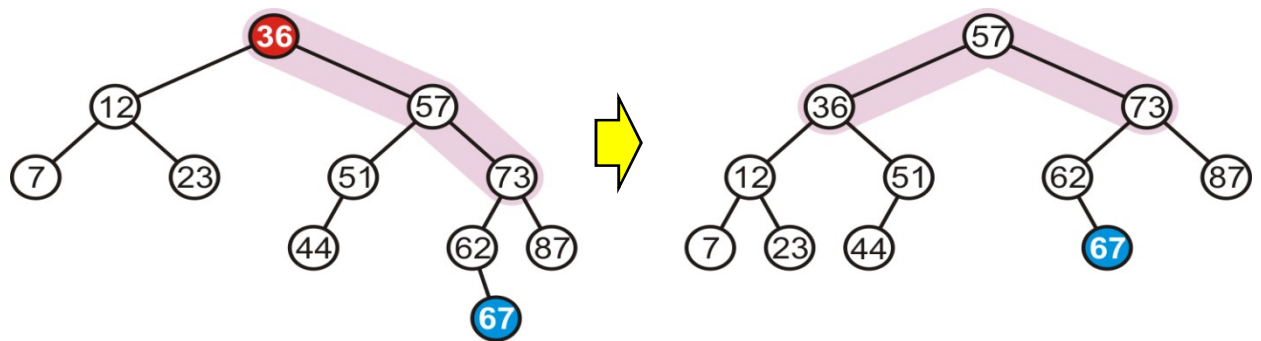


Maintaining balance: Two symmetric cases

There are two symmetric cases to those we have examined:

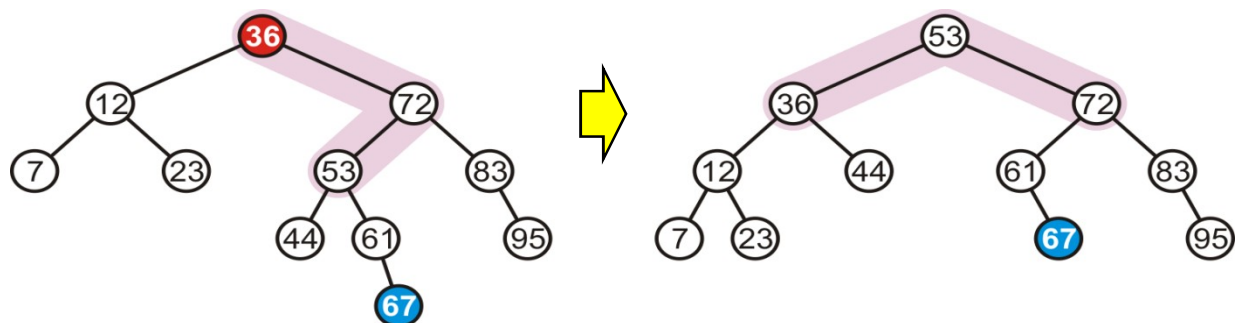
- ❑ Insertions into the right-right sub-tree

- ❖ *This can be solved using Left Rotation*



- ❑ Insertions into either the right-left sub-tree

- ❖ *This can be solved using Right-Left Rotation*



Maintaining Balance: Erase / Deletion

- ❑ Removing a node from an AVL tree may cause more than one AVL imbalance
 - Like insert, erase must check after it has been successfully called on a child to see if it caused an imbalance
 - Unfortunately, it may cause $O(h)$ imbalances that must be corrected
 - ✓ Insertions will only cause one imbalance that must be fixed
 - The movement of trees, however, may require that more than one node within the triplet has its height corrected
-

Height of an AVL Tree

- ❑ By the definition of complete trees, any complete binary search tree is an AVL tree
- ❑ Thus an upper bound on the number of nodes in an AVL tree of height h is a perfect binary tree with $(2^{h+1} - 1)$ nodes
 - What is an lower bound?

❖ Let $F(h)$ be the fewest number of nodes in a tree of height h

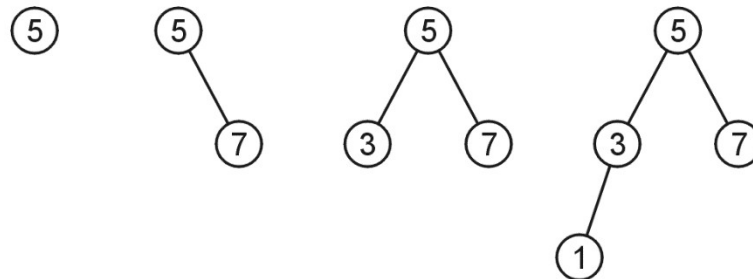
❖ We know that :

$$F(0) = 1$$

$$F(1) = 2$$

$$F(2) = 4$$

...



➤ Can we find $F(h)$?

Height of an AVL Tree

Contd

- For $h \geq 3$, an AVL tree of height h contains the root node, one AVL subtree of $(h-1)$ and the other AVL subtree of height $(h-1)$ or $(h-2)$
- However, for minimum number of nodes in an AVL tree (the smallest possible / worst-case AVL tree) of height h would have:
 - ✓ A worst-case AVL tree of height $(h-1)$ on one side,
 - ✓ A worst-case AVL tree of height $(h-2)$ on the other, and
 - ✓ The root node
 - ❖ We get: $F(h) = F(h-1) + F(h-2) + 1$
 - ❖ This is a recurrence relation:

$$F(h) = \begin{cases} 1 & h = 0 \\ 2 & h = 1 \\ F(h-1) + F(h-2) + 1 & h > 1 \end{cases}$$

Height of an AVL Tree

Contd

□ Now, we know that $F(h-1) > F(h-2)$, as height of a tree grows, number of nodes cannot reduce.

$$\begin{aligned} \text{➤ } F(h) &= F(h-1) + F(h-2) + 1 > 2F(h-2) = 2(F(h-3) + F(h-4) + 1) \\ &> 4F(h-4) = 4(F(h-5) + F(h-6) + 1) \\ &> 8F(h-6) > 16F(h-8) > \dots > 2^i F(h-2i) \end{aligned}$$

✓ When we pick $i = (h/2 - 1)$, and let's assume i is an integer, when h is even

$$\text{➤ } F(h) > 2^i F(h-2i) = 2^{(h/2 - 1)} F(h-2(h/2 - 1)) = 2^{(h/2 - 1)} F(2) = 2^{(h/2 - 1)} 2 = 2^{h/2}$$

❖ So an AVL tree with height h will have at least $2^{h/2}$ nodes

➤ Taking logarithm $h < 2\log(F(h))$

➤ Recall that : $F(h)$ be the fewest number of nodes in a tree of height h

➤ Hence, $n \geq F(h)$, where n is total number nodes in the AVL tree

➤ Therefore, $h < 2\log(n)$

❖ Thus the height of an AVL tree is $O(\log n)$

Acknowledgement

Most slides courtesy : Douglas Wilhelm Harder,
MMath, UWaterloo
